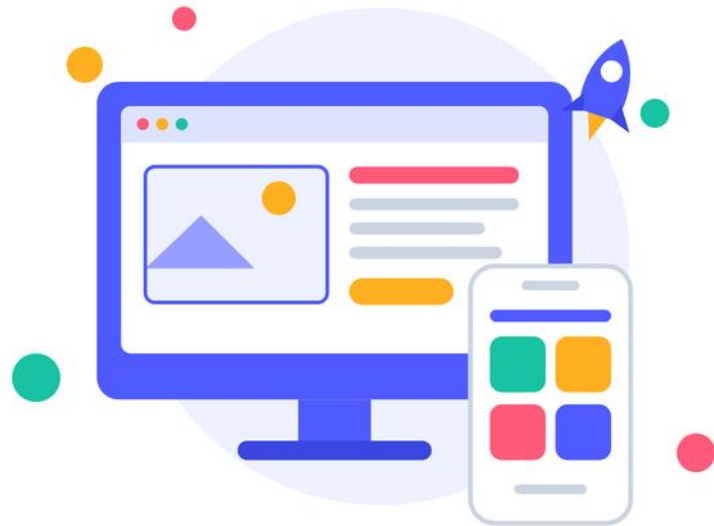


WEEK 03 · CSS FOUNDATIONS · CASCADE · BOX · TOKENS

MAKING IT LOOK LIKE A DECISION

Two weeks of markup, and one file today — after which it stops looking like a document.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Without CSS, every website would look plain, like a Word document.

FROM THIS COURSE, BEFORE YOU WERE IN IT

WHERE WE ARE

Week 3 of 12. Your week-2 page is in your repo; today it gets a stylesheet.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: attach and lose · box, table, form · type, colour and states · one question to Claude · values to tokens. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

Weeks 4, 5 and 6 are layout on top of this. Tailwind is these properties, shorter.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment — week 2's page, byte for byte, plus one file

↶ reload

open ↷

[Skip to content](#)

Maya Cohen

Third-year student at Ruppín. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 - from 700px

[Home](#)[About me](#)[What I am learning](#)[Contact me](#)[Skip to content](#)

Maya Cohen

Third-year student at Ruppín. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 - from 700px

[Home](#)[About me](#)[What I am learning](#)[Contact me](#)[Contact me](#)[About me](#)

targets/week-03.png · 1280

[Skip to content](#)

Maya Cohen

Third-year student at Ruppín. I already write C# and Python; this semester I am finding out what the browser does with the text I

375 · same file, nothing overflows

► **THE POINT** — the markup did not change. Not one element. The whole difference is `styles.css`: a reset, a token set, a type scale, a colour system that passes contrast, hover and focus states, a table and a form that look finished, and a dark theme in one block. You type every piece of it today: once in a cycle, once in the assignment.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1-5 · PRIVATE TUTOR

ALLOWED

Ask it why a rule loses, what `rem` measures from, what `border-collapse` changes. Ask it to review a stylesheet *you* typed. Then check the answer in DevTools — the Styles pane, the box diagram, the colour picker's ratio — not in your gut.

NOT ALLOWED

Pasting generated CSS into the assignment. **You type every character.** The tell this week: "quickest fix: `#page-title.title`" (or `!important`). It works. Cycle 4 shows you what it costs, and what the answer put in a footnote.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one question, twice, in whatever chat you have.

CYCLE 1 · 25 MIN

ATTACH, AND THE RULE THAT LOSES

The film club page from last week gets its first stylesheet. One link, one rule — then a rule that is written last, matches, and still loses. The three real reasons "nothing happened", and the four steps that decide who wins.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/01-attach-and-lose · link + h1 · a <style> block moved above and below the link · body font that the button refuses · nav ul li a, then a flat a

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 1 · Network black, Styles pane with nothing struck through

3 YOU DO

8 MIN

warmup/ — four rules that lose, one stylesheet, no !important, no new selectors

4 DEBRIEF

2 MIN

"I wrote it last" is step four · (0,0,4) vs (0,0,1) · ↗ reference

YOU DO · AI-FREE

CYCLE 1 · BEFORE YOU TYPE

"I CHANGED THE CSS AND NOTHING HAPPENED"

demos/02-cascade/01-conflict - three rules, all losing

↺ reload

open ↗

This heading is supposed to be teal

This callout is supposed to be blue. Someone edited it. Twice.

Everything in this box is red, because the box says so.

Everything except this button:

Order a flat white

► **HANDS-ON** [demos/02-cascade/01-conflict/](#) — Three symptoms. Do not explain them. Ask which one they would debug first.

THE CASCADE IS AN ALGORITHM. HERE IT IS.

1 Origin

browser default, then yours. Yours wins.

2 Importance

!important jumps the queue. Almost never the answer.

3 Specificity

a triple — (a, b, c) — compared column by column.

4 Source order

last one written wins. Only reached on an exact tie.

There is no fifth step. If two rules are still tied after source order, they are the same rule.

SPECIFICITY IS NOT A COUNT

HOW EVERYONE LEARNS IT

"More selectors means stronger. Count them."

`.site .nav .list .item a` — five pieces, so surely it beats `#cta`, which is one.

WHAT IT ACTUALLY IS

Three numbers, compared like a version number.

`#cta` is `(1, 0, 0)`.

`.site .nav .list .item a` is `(0, 4, 1)`.

1.0.0 beats 0.4.1. It is not close, and it never will be.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

[Skip to content](#)

Ruppin Film Club

One film a week, in the small hall, for anyone on campus.

Ruppin Film Club

WIDE 640x160 - shown from 700px



- [Home](#)
- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Screenings this month

Film	Date	Room
Alien	Tuesday 2 November, 19:30	204
The Apartment	Tuesday 9 November, 19:30	204
Spirited Away	Tuesday 16 November, 19:30	Auditorium

► [How screenings work](#)

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Compare — `live-code/01-attach-and-lose/checkpoint.png` is this picture at 1280; `live-code/01-attach-and-lose/end/` is the code behind it. Network: `styles.css` black, 200. Styles pane on a nav link: one rule, nothing struck through. The Send button is still in the browser's font — on purpose, until cycle 2.

CYCLE 2 · 25 MIN

THE BOX, THE TABLE AND THE FORM

A measure for the page, then **width: 100%** on the form fields — and they escape the fieldset. Three lines at the top fix every box on the page. Then the two lines a table and a form cannot do without, and the vertical margins that refuse to add up.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/02-box-table-form · max-width: 65ch + margin: 0 auto · width: 100% that overflows · box-sizing: border-box · [type="radio"] · font: inherit · border-collapse: collapse · margins that collapse

2 WE DO

5 MIN

you type it on your copy, the three lines FIRST · compare with checkpoint 2 · the box diagram includes the padding

3 YOU DO

8 MIN

your own week-03 page: the reset, the measure, the table, font: inherit, width: 100% · AI as tutor, checked in the box diagram

4 DEBRIEF

2 MIN

344 → 300 · why the button did not inherit · ↗ reference

THREE LINES, AT THE TOP OF EVERY STYLESHEET

THE DEFAULT

```
.card {  
  width: 240px;  
  padding: 20px;  
  border: 2px solid;  
}  
/* on the page: 284px */
```

`width` means the CONTENT. Padding and border are added on.

BORDER-BOX

```
*,  
*::before,  
*::after {  
  box-sizing: border-box;  
}  
/* on the page: 240px */
```

`width` means the whole box. Which is what everyone thought it meant.

INHERITANCE: SOME THINGS FALL DOWN THE TREE

INHERITS

`color` `font-family` `font-size`
`line-height` `text-align`
`visibility`

Anything about TEXT. Set it once on `body` and the page is styled.

DOES NOT INHERIT

`margin` `padding` `border`
`background` `width` `display`

Anything about the BOX. Imagine if padding inherited — every nested element would nest its own padding.

THE EXCEPTION THAT BITES

Form controls — `input`, `button`, `textarea` — have their OWN font and colour from the browser, and a set value beats an inherited one.

`font: inherit` is how you opt back in.

CYCLE 2 · THE TWO LINES

TWO COMPONENTS, AND THE LINE EACH ONE CANNOT DO WITHOUT

THE TABLE

```
table { border-collapse: collapse; }

th, td {
  padding: 0.5rem 0.75rem;
  border-bottom: 1px solid hsl(170 20% 70%);
  text-align: left;
}
```

Without `collapse` every cell keeps its own border and each line between rows is drawn twice. `th` is centred by default.

THE FORM

```
input, select, textarea, button { font: inherit; }

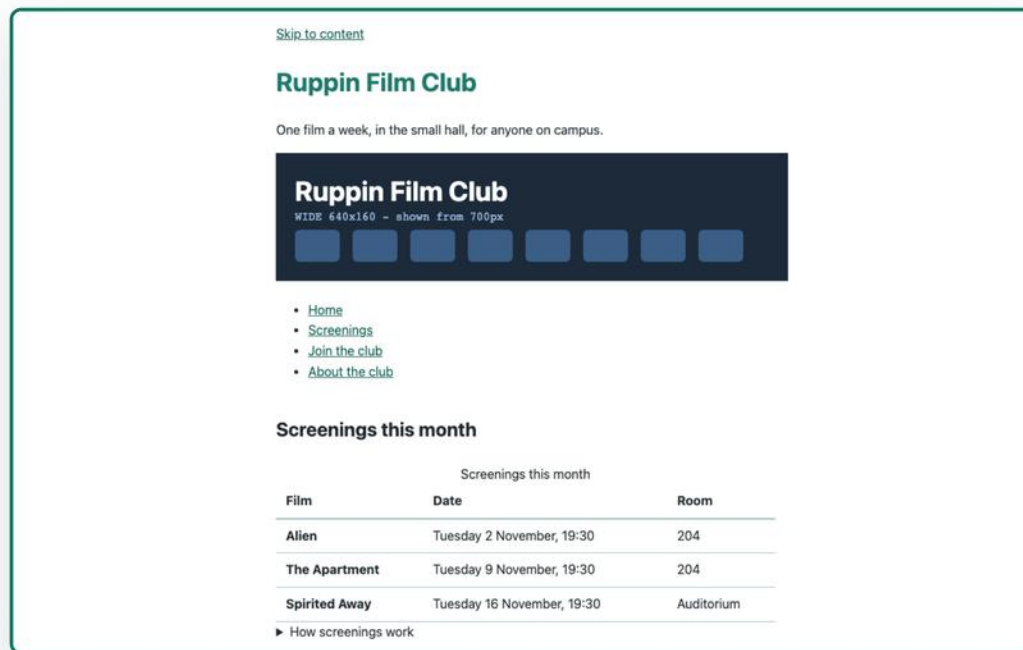
input, select, textarea { width: 100%; }

input[type="radio"] { width: auto; }
```

Controls get their own font from the browser; `font: inherit` opts back in. `width: 100%` drops a field under its label without `display` — and the attribute selector undoes it for the radios.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-box-table-form/checkpoint.png` is this picture at 1280; `live-code/02-box-table-form/end/` is the code behind it. Scroll to the form: the fields sit inside the fieldset, the radios are small, Send is in the page's font. In the box diagram of a field, the width includes the padding.

CYCLE 3 · 30 MIN

TYPE, COLOUR, AND THE PAGE ANSWERING BACK

A type scale in `rem` that follows the reader's own setting. One hue in `hsl`, four lightnesses — and a muted grey that "looks subtle" and fails at 2.55:1 until the third number moves. Then links that answer: colour, hover, and the focus ring somebody was about to delete.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/03-type-colour-states · 1.0625rem and $\times 1.25$ · `hsl(170 ...)` at four lightnesses, the ratio in the colour picker · a, a:hover · outline: none, then :focus-visible with outline + outline-offset · [aria-current] · the button

2 WE DO

8 MIN

you type it on your copy · compare with checkpoint 3 · Tab from the top: a ring at every stop; every pair ≥ 4.5 in the picker

3 YOU DO

10 MIN

your own page: your hue, your scale, your ring — checked with Tab and the contrast checker

4 DEBRIEF

2 MIN

:focus vs :focus-visible on a click · the hsl trap · ↗ reference

A RULE FOR EVERY UNIT

REM

Type sizes and spacing. Your default for almost everything. Follows the reader's own font-size setting.

EM

Only when a value should scale with THIS element's text — padding inside a button that comes in three sizes.

%

Widths, relative to the containing block. `width: 100%` is the one you will use today.

CH

Measure, and nothing else.
`max-width: 65ch`.

PX

Hairlines and edges: borders, outlines, small radii. Things that should NOT scale.

VW / VH

Viewport-sized things only.
`font-size: 4vw` is unreadable on a phone and absurd on a monitor. Sparingly.

CONTRAST IS A NUMBER, AND YOU CAN CHECK IT

4.5:1

body text. The line most designs fail.

3:1

large text — 24px, or 18.66px bold — and meaningful UI edges.

7:1

AAA. Nice, rarely required.

Two colours in, one ratio out. Thirty seconds a pair, and it is the difference between a page that looks nice on your laptop and a page that can be read on a projector, in sunlight, or by someone with low vision.

Tools worth keeping:

webaim.org/resources/contrastchecker ·

color.adobe.com · colorhunt.co



ATTRIBUTES, STATES, AND THINGS THAT ARE NOT THERE

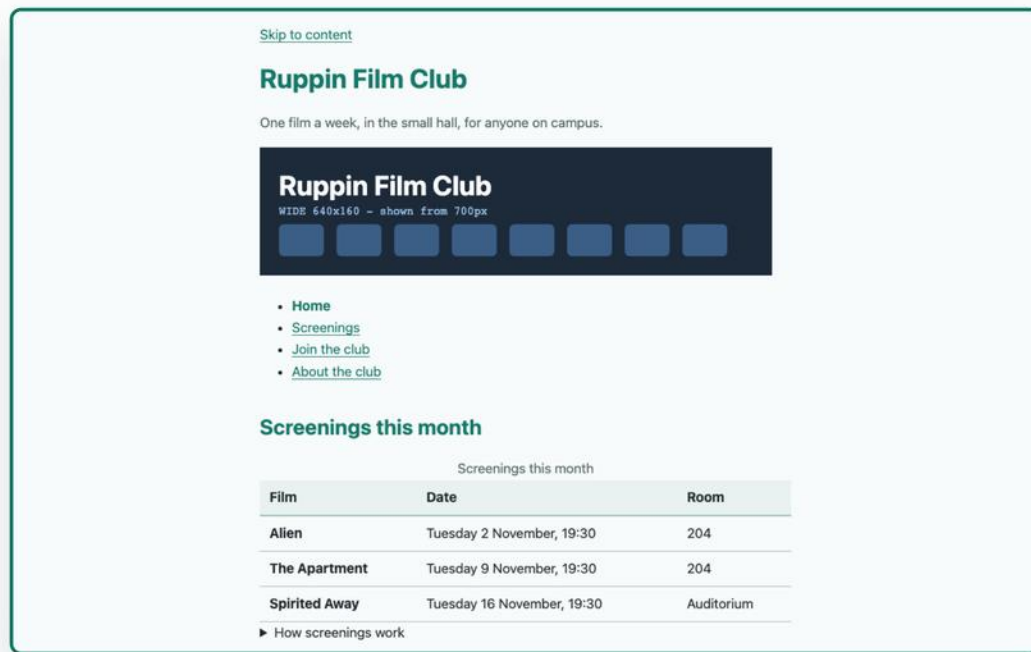
```
1 /* attribute: no class needed */
2 input[type='radio'] { width: auto; }
3 a[aria-current='page'] { font-weight: 700; }
4
5 /* state: the ring that takes no space */
6 a:focus-visible,
7 button:focus-visible {
8     outline: 3px solid hsl(170 70% 28%);
9     outline-offset: 2px;
10 }
11
12 /* position among siblings */
13 tbody tr:nth-child(even) { ... }
```

`:focus-visible` rather than `:focus`: the browser decides whether the user is navigating by keyboard, and it is better at that than you are. `outline` rather than `border`: it takes no space, so nothing moves; `outline-offset` lifts it off a control that already has a border. `outline: none` with nothing put back is a fail in the assignment.

`::before` and `::after` create a box that is not in the HTML. They need a `content` property to exist at all — even `content: ''`.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-type-colour-states/checkpoint.png` is this picture at 1280; `live-code/03-type-colour-states/end/` is the code behind it. The real difference is not in the pixels: Tab from the address bar shows a ring at every stop, the colour picker shows 4.5 or more on every pair, and Chrome's "Very large" font setting scales the whole page. These are the grader's `link-states`, `type-scale` and `contrast-aa` rows.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and DevTools on the Styles pane.

CYCLE 4 · 25 MIN

"WHY DOESN'T MY RULE APPLY — AND WHAT'S THE QUICKEST FIX?"

A `.title` rule that is last in the file and still loses to a tutorial's `#page-title`. What comes back explains specificity exactly right — and fixes it in the wrong direction.

1 I DO

7 MIN

2 on the next slide, 5 in the chat · ai-lab/04-why-doesnt-my-rule-apply · P1 live, then RUN it: Styles pane, Rendering in dark, a class rule appended at the end · transcript.md if the room got something else

2 WE DO

6 MIN

everyone pastes P1 · run what came back · find what the fix did to the next class rule, before it is named

3 YOU DO

10 MIN

write P2 as a checkable constraint · run it in both schemes · one question about your own stylesheet · the journal line

4 DEBRIEF

2 MIN

"quickest" got the quick answer; the right one was a footnote · ↗ the journal exemplar

YOU DO · AI AS TUTOR

➤ REFERENCE · 16, 19, 47

CYCLE 4 · THE GAP

WHAT TO DO WHEN YOUR RULE LOSES

THE REFLEX

```
.title {  
  color: teal !important;  
}
```

Works in three seconds. So does `#page-title.title` at (1,1,0) — the "quickest fix". Both raise the loser, and every class rule you write afterwards loses the same way. Measured: the dark-scheme `.title` that was already in the file.

THE FIX

```
/* was: #page-title { color: grey } */  
.page-title {  
  color: grey;  
}
```

Lower the winner instead of raising the loser — to a class, or to `h1` (0,0,1). The id stays in the HTML as an anchor; the stylesheet just stops selecting by it. Now everything is overridable, including the rule in the dark block.

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you **run** the answer to P1 — Styles pane, then Rendering → dark, then a `.title` rule appended at the end — instead of stopping when the heading changed colour?
2. Does your P2 state a constraint DevTools can check — "the winning selector goes down", "no `!important`", "the HTML does not change" — and does what came back meet it in both schemes?
3. Does your journal line name **a tool and a measurement** — "Styles: `#page-title` struck through", "dark: heading `rgb(21, 121, 105)`, links `rgb(133, 224, 209)`" — and not just "Claude was right / wrong"?

Compare — `ai-lab/04-why-doesnt-my-rule-apply/checkpoint-he.md` has the same three questions and the measurements; `artifact/claude-p1/` and `artifact/claude-p2/` are the two stylesheets, ready to open side by side.

CYCLE 5 · 20 MIN

"THE CLIENT WANTS IT PURPLE": FROM VALUES TO TOKENS

The hue 170 is written fourteen times in the file. One `:root` block later it is written once, the page has not moved a pixel, and purple is one edit — and a dark theme costs one more block.

1 I DO

7 MIN

2 on the next two slides, 5 in the editor · live-code/05-purple-to-tokens · `:root`, four families · `--hue: 280` · a misspelt token that fails in silence · the dark block · `--duration`

2 WE DO

5 MIN

you type it on your copy · checkpoint 5 is checkpoint 3, on purpose · `Ctrl+F "170"`: one hit

3 YOU DO

6 MIN

your own page: your colours into `:root`, the dark block · AI as tutor on the lighter accent

4 DEBRIEF

2 MIN

the token that failed in silence · reduced motion without !important

YOU DO · AI AS TUTOR

CUSTOM PROPERTIES

```
1  :root {  
2    --color-accent: hsl(258 72% 42%);  
3    --space-4: 1rem;  
4  }  
5  
6  a {  
7    color: var(--color-accent);  
8    padding: var(--space-4);  
9  }  
10  
11 /* a fallback, if the token was never defined */  
12 color: var(--color-accent, hotpink);
```

Two dashes to declare, `var()` to read. That is the whole feature.

FOUR FAMILIES. THAT IS THE WHOLE SYSTEM.

`--color--*`

bg · surface · border · text
· text-muted · accent

`--space--*`

a scale in rem: 1 · 2 · 3 · 4 ·
5 · 6. Steps, never one-off
numbers.

`--radius--*`

sm · md · lg · full. Two is
enough to start.

`--font--*`

the stack, plus the size
scale from act 5.

Name them for the JOB, never for the value. `--color-accent`, not
`--color-purple` — because the day the accent stops being purple, a
stylesheet full of `--color-purple: green` is unreadable.



CYCLE 5 · ONE BLOCK

A DARK THEME IS ONE BLOCK, AND ZERO RULES BELOW IT

```
1 @media (prefers-color-scheme: dark) {  
2   :root {  
3     --color-paper: hsl(var(--hue) 20% 10%);  
4     --color-ink: hsl(200 20% 92%);  
5     --color-brand: hsl(var(--hue) 60% 70%);  
6     /* ...the same names, every one of them */  
7   }  
8 }  
9 /* nothing below :root changes. Count it. */
```

The same token names, different values. Every rule that says `var(--color-ink)` picks up the new one; if a rule below the block has to change, a literal colour is still hiding in it.

The accent gets lighter. Contrast is a relationship, not a property of a colour: `hsl(170 70% 28%)` reads at 5.08:1 on near-white and 3.19:1 on near-black. Copying the same accent into both blocks is the usual way to fail this row.

Test it without touching your OS: DevTools → Rendering → *Emulate CSS media feature prefers-color-scheme*.

A TOKEN CAN HOLD ANYTHING, INCLUDING TIME

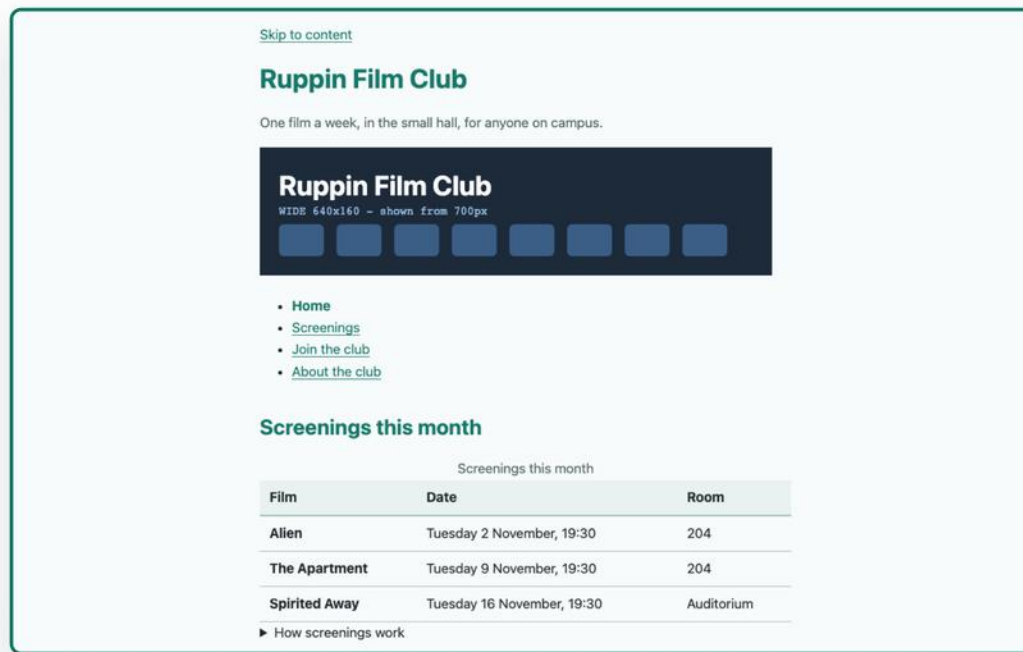
```
1  :root {  
2    --duration: 150ms;  
3  }  
4  
5  a { transition: color var(--duration) ease; }  
6  
7  /* some people get motion sickness from moving interfaces */  
8  @media (prefers-reduced-motion: reduce) {  
9    :root {  
10     --duration: 0s;  
11   }  
12 }
```

The usual version of this forces `transition: none !important` on every element. With a token it is one value, and no override war.



CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-purple-to-tokens/checkpoint.png` is this picture at 1280, and it is checkpoint 3 again, on purpose; `live-code/05-purple-to-tokens/end/` is the code behind it. The difference: `Ctrl+F "170"` finds one hit; change it and the page recolours; Rendering → dark gives a dark page with a light accent; reduced motion gives 0s. These are the grader's `dark-theme` and `motion-safe` rows.

ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Your week-2 page in `week-03/`, one `<link>`, and `styles.css`: the three lines (cycle 2) · 5
`--color-*, 5 --space-*, 2`
`--radius-*, 3 --font-*` (cycle 5) · the ground on `body` with a measure in `ch` (cycle 2) · a type scale, tight heading leading, spacing from the scale, `a:hover` and `a:focus-visible` with an `outline` (cycle 3). Two commits.

EXTEND · STRETCH AND CHALLENGE

The table with `border-collapse`, the form with `font: inherit` (cycle 2) · `about.html` from the same file · every pair at 4.5:1 (cycle 3) · `::before` or an attribute selector (cycles 2, 3). Then: a dark theme through tokens only, a transition that respects `prefers-reduced-motion`, no literal colour below `:root` (cycle 5).

THE RULES, AND THE EXAM

No Flexbox, Grid, positioning, !important or style="".

Normal flow, on purpose — weeks 4 and 5 sit on it. AI as tutor: it explains, you type, you check in DevTools. Repo URL and SHA in Moodle. Home assignment 1 is out today. Week 13 asks about this code, offline.

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

1. `#cta` against `.site .nav .list .item a`. Which wins, and what are the two triples?
2. An element has `width: 300px`, `padding: 20px` and a `2px` border. How wide is it on the page? Now add the three lines and answer again.
3. You want the whole page one hue cooler. In a stylesheet with tokens, how many lines do you change? In one without?

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: every selector on one page, the cascade with its layers, the inheritance table, `em` against `rem` in numbers, colour syntax, the WCAG rows, the box-sizing story, margin collapsing in full, logical properties, the two media queries with `color-scheme` and `light-dark()`. The next slide lists it.

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · The CSS catalogue](#) ← spine 30

[7 · One thing you may have been told, that is wrong](#) ← spine 4

[9 · The cascade in full — CSS Cascade Level 5](#) ← spine 4

[11 · Compute it yourself](#) ← spine 4

[12 · Specificity, worked](#) ← spine 4

[15 · What inherits, and what does not](#) ← spine 9

[16 · Every selector, on one page](#) ← spine 20

[19 · Grouping, and how flat should a selector be](#) ← spine 20

[23 · box-sizing: a history, and why the default is the wrong one](#) ← spine 9

[24 · Margins collapse. This is not a bug.](#) ← spine 9

[27 · The box, and its shorthands](#) ← spine 9

[31 · The trap inside hsl](#) ← spine 14

[32 · Colour](#) ← spine 14

[33 · Contrast, and where to check it](#) ← spine 14

[40 · em and rem, the arithmetic](#) ← spine 14

[41 · Units](#) ← spine 14

[43 · The payoff, and it is bigger than colour](#) ← spine 23

[45 · Tokens: the convention this course uses](#) ← spine 23

[46 · Dark and reduced motion: the two queries, color-scheme, and light-dark\(\)](#) ← spine 23

[47 · The five that will cost you time](#) ← spine 20